

Assignment on Advanced Python

Instructions

- Read all questions carefully before starting.
- Only Python's standard library, NumPy, Pandas, and SciPy may be used.
- Write efficient and readable code.
- Follow the prescribed file structure and input/output formats exactly. Formatting mistakes may lead to mark deductions and will not be considered for re-evaluation.
- LLMs and AI tools may be used, but the primary objective of this assignment is your learning, so use them critically and wisely.

Submission Format

```
<EO ID>/
|-- q1/
|   |-- q1_t1.py
|   |-- q1_t2.py
|   |-- q1_t3.py
|   '-- q1_t4.py
|-- q2/
|   '-- projectile.py
'-- q3/
    '-- delivery_hub.py
```

Question 1: The Lost Research Archive (40 marks)

You have joined a team responsible for recovering data from an old research archive. The archive consists of several datasets that were processed by different systems over many years. Your job is to reconstruct missing analyses and generate the final outputs required by the archive team.

Each task represents a different stage in the recovery process. The datasets for each subpart are independent.

Task 1: Passenger Archive Analysis (14 marks)

The first dataset recovered from the archive contains passenger records from a historic voyage. Before the archive can be restored, the team requires a detailed demographic survival report.

Write a script named `q1_t1.py` that reads `q1_t1_input.csv` and writes the result to `q1_t1_output.csv`.

Part A: Warmup

For every combination of passenger class (`pclass`) and sex (`sex`), compute the following statistics:

- `passenger_count`: Number of passengers in the group.
- `survivor_count`: Number of passengers with `survived = 1`.
- `survival_pct`: Survival percentage, computed as

$$100 \times \text{mean}(\text{survived})$$
- `avg_age`: Average passenger age.
- `median_fare`: Median fare paid by passengers in the group.
- `age_std`: Standard deviation of passenger age.

Part B: Class Ranking

Within each passenger class, rank all groups according to their survival percentage using **dense ranking**.

For example:

90% $\rightarrow$ 1, 90% $\rightarrow$ 1, 75% $\rightarrow$ 2, 40% $\rightarrow$ 3

Store the result in a column named `survival_rank`.

Part C: Rolling Survival Trend

Within each passenger class (`pclass`), sort passengers in ascending order of `fare`.

For every passenger, compute a rolling average of the `survived` column using a window size of 5 within the corresponding passenger class. Let this value be denoted by

`rolling_survival`

The rolling average should be computed using all available observations when fewer than 5 preceding observations exist.

After computing `rolling_survival`, calculate the following statistics for every combination of `pclass` and `sex`:

- `mean_rolling_survival`: Mean of `rolling_survival` values in the group.
- `max_rolling_survival`: Maximum `rolling_survival` value in the group.

These two columns must also be included in the final output.

The complete output CSV must therefore contain the following columns:

```
pclass,
sex,
passenger_count,
survivor_count,
survival_pct,
avg_age,
median_fare,
age_std,
survival_rank,
mean_rolling_survival,
max_rolling_survival
```

Task 2: Signal Recovery Puzzle (8 marks)

A second dataset contains measurements from a legacy sensor. Unfortunately, calibration resets were not logged directly.

The value 0 in column `signal` represents a calibration reset. For every row, determine how many observations have occurred since the most recent reset.

If no previous reset exists, count from the beginning of the dataset.

Write a script named `q1_t2.py` that reads `q1_t2_input.csv`, creates a new column named `since_reset`, preserves all existing rows and columns, and writes the result to `q1_t2_output.csv`.

Example

Given:

```
signal = [7, 2, 0, 3, 4, 2, 5, 0, 3, 4]
```

the corresponding values should be:

```
since_reset = [1, 2, 0, 1, 2, 3, 4, 0, 1, 2]
```

Task 3: Fragment Reconstruction Puzzle (8 marks)

A third dataset was reconstructed from multiple experimental groups. Some measurements were lost during transmission and appear as missing values.

Write a script named `q1_t3.py` that reads `q1_t3_input.csv`. Compute a group-wise rolling average of the `measurement` column, grouping by `experiment` while preserving the original order of observations within each group.

Use a rolling window size of 3 and store the result in a new column named `smoothened_measurement`. The completed dataframe should be written to `q1_t3_output.csv`.

Example

Consider the following input:

experiment	measurement
Project Atlas	10.0
Project Nova	5.0
Project Atlas	NaN
Project Orion	12.0
Project Nova	7.0
Project Helios	6.0
Project Atlas	4.0
Project Orion	NaN
Project Nova	9.0
Project Helios	3.0
Project Atlas	8.0
Project Orion	6.0
Project Nova	NaN
Project Helios	4.0
Project Atlas	2.0
Project Orion	15.0

Using a rolling window of size 3 within each experiment, the expected output is:

experiment	measurement	smoothened_measurement
Project Atlas	10.0	10.000000
Project Nova	5.0	5.000000
Project Atlas	NaN	10.000000
Project Orion	12.0	12.000000
Project Nova	7.0	6.000000
Project Helios	6.0	6.000000
Project Atlas	4.0	7.000000
Project Orion	NaN	12.000000
Project Nova	9.0	7.000000
Project Helios	3.0	4.500000
Project Atlas	8.0	6.000000
Project Orion	6.0	9.000000
Project Nova	NaN	8.000000
Project Helios	4.0	4.333333
Project Atlas	2.0	4.666667
Project Orion	15.0	10.500000

Task 4: Minefield Mapping System (10 marks)

The final archive contains coordinates from an old exploration robot. The robot operated on a grid containing hidden hazards.

Each row in `q1_t4_input.csv` describes a grid location and whether a hazard is present.

Write a script named `q1_t4.py` that reads `q1_t4_input.csv`, reconstructs the robot's hazard map, and writes the final grid representation to `q1_t4_output.csv`.

For every non-hazard cell, compute the number of hazards present in the eight neighboring cells:

$$\begin{aligned}
 &(x-1, y-1), (x-1, y), (x-1, y+1), \\
 &(x, y-1), (x, y+1), \\
 &(x+1, y-1), (x+1, y), (x+1, y+1)
 \end{aligned}$$

Cells outside the grid should be ignored.

If a location contains a hazard, its value should be recorded as `NaN`.

After computing these values, transform the data into a grid representation where:

- Rows correspond to `y` coordinates.
- Columns correspond to `x` coordinates.
- Entries contain the adjacent hazard counts.

The archive restoration process is complete once all four output files have been generated successfully.

Question 2: Asteroid Defense System (40 marks)

Problem Statement

A mysterious portal has opened above Earth. From it emerged an **Arcane Asteroid** – a magical object from another world. Unlike ordinary asteroids, it does not follow the laws of

classical mechanics. Scientists have observed that its motion is influenced by unknown magical forces, making its future trajectory difficult to predict. For a brief period, Earth's observatories were able to track the asteroid. Unfortunately, the portal emits powerful interference that disrupts radar systems. As a result:

- observations contain measurement noise,
- observations occur at irregular timestamps,
- contact with the asteroid is lost after only a short portion of its flight.

The asteroid is still heading toward Earth. Humanity's last hope is the **Aegis Interceptor**, a high-speed defense projectile launched from a planetary defense station located at $(0,0)$. The interceptor launch parameters have already been selected by mission control. Your task is to determine whether the interceptor successfully destroys the asteroid. The fate of humanity depends on your calculations.

Input Format

The observatory provides measurements in the following format:

```
N K
t1 x1 y1
t2 x2 y2
...
tN xN yN
ANGLE SPEED
```

where:

- N is the number of asteroid observations.
- K is the interceptor drag coefficient.
- t_i is the observation timestamp.
- (x_i, y_i) is the observed asteroid position.
- **ANGLE** is the interceptor launch angle in degrees.
- **SPEED** is the interceptor launch speed.

Example:

```
10 0.015
0.0 5.13 120.41
0.7 9.22 119.03
1.9 16.61 114.18
2.8 22.03 109.77
4.0 30.20 101.62
5.1 36.97 92.41
6.3 44.62 80.14
7.2 50.58 70.95
8.6 59.12 54.22
9.8 67.21 36.74
42.0 120.0
```

Let $t_{\text{obs}} = t_N$ denote the timestamp of the final observation. Immediately after receiving the final observation, the defense system launches the interceptor.

Asteroid Model

The asteroid's trajectory can be approximated by:

$$\begin{aligned}x_A(t) &= a + bt + ct^2 \\ y_A(t) &= d + et + ft^2\end{aligned}$$

for unknown coefficients. Because the observations contain noise, the coefficients must be estimated from the provided data. The asteroid continues moving according to these equations after the final observation.

Interceptor Model

The interceptor launches from $(0,0)$ at time t_{obs} . Let (x_I, y_I) denote the interceptor position and (v_x, v_y) denote its velocity. Its motion is governed by:

$$\begin{aligned}\frac{dx_I}{dt} &= v_x \\ \frac{dy_I}{dt} &= v_y \\ \frac{dv_x}{dt} &= -K v_x \sqrt{v_x^2 + v_y^2} \\ \frac{dv_y}{dt} &= -g - K v_y \sqrt{v_x^2 + v_y^2}\end{aligned}$$

where $g = 9.81$. The interceptor is launched with:

$$\begin{aligned}v_x(t_{\text{obs}}) &= \text{SPEED} \cos(\text{ANGLE}) \\ v_y(t_{\text{obs}}) &= \text{SPEED} \sin(\text{ANGLE})\end{aligned}$$

where **ANGLE** is specified in degrees.

Interception Condition

At some future time $T \geq t_{\text{obs}}$, the interceptor destroys the asteroid if:

$$\|A(T) - I(T)\|_2 \leq 1$$

where:

- $A(T)$ is the asteroid position.
- $I(T)$ is the interceptor position.

The value 1 represents the interceptor's effective blast radius.

Task

Using only the provided observations:

1. Estimate the asteroid trajectory.
2. Predict its future motion.
3. Simulate the interceptor trajectory.
4. Determine whether the interceptor successfully intercepts the asteroid.
5. If an interception occurs, report the earliest interception time.

Output Format

If the interceptor successfully destroys the asteroid, print:

HIT TIME

where TIME is the earliest interception time. Example: HIT 13.482

If no interception occurs, print:

MISS

All reported times must have an absolute error of at most 10^{-3} .

Constraints

```
5 <= N <= 100
0 <= ti <= 100
ti < ti+1
0.001 <= K <= 0.1
0 <= ANGLE <= 90
1 <= SPEED <= 500
```

Hidden test cases may contain:

- Varying drag coefficients.
- Different noise levels.
- Different asteroid trajectories.
- Observations covering different portions of the asteroid's flight.

Your solution should not rely on hardcoded assumptions about the trajectory parameters.

Notes

- The observations are noisy; fitting the trajectory exactly is not expected.
- The asteroid trajectory must be inferred from the provided data.
- The interceptor trajectory is governed by differential equations and will require numerical integration.
- Numerical methods from SciPy may be useful for fitting and simulation.

Question 3: Optimizing Delivery Hubs (40 marks)

A rapidly growing e-commerce company needs to deploy K new regional delivery hubs across a major metropolitan area. They have provided you with a large dataset containing the 2D GPS coordinates (Longitude and Latitude) of millions of recent delivery destinations, stored in a comma-separated text file (`delivery_locations.txt`).

Your task is to implement the K-Means clustering algorithm to group these delivery locations into K optimal zones, minimizing the travel distance from the center of each zone (the hub) to its respective destinations.

A Primer on the K-Means Algorithm (Hub Optimization): To find the optimal hub locations, the algorithm alternates between two steps until the locations stabilize:

1. **Initialization:** Select K delivery locations from the dataset to act as the initial, temporary hub coordinates.
2. **The Assignment Step:** Calculate the straight-line distance from every delivery location to every hub. Assign each delivery location to the zone of its strictly nearest hub.
3. **The Update Step:** Calculate the geographic center (the mean coordinates) of all delivery locations assigned to a specific zone. Move the hub to this new center coordinate.
4. **Convergence:** Repeat the Assignment and Update steps until the hub locations no longer change (i.e., the zone labels remain identical to the previous iteration).

The Performance Constraint: To ensure the algorithm can scale to process millions of real-time coordinates, your code must be hyper-optimized. You are **strictly forbidden** from using `for` loops, list comprehensions, or any iterative structures over the dataset or the clusters. The only permitted loop is the main `while` loop controlling the convergence iterations. Everything else must be purely vectorized `numpy` operations.

Task: Implement the complete K-Means algorithm by defining the following functions strictly without `for` loops:

1. `load_data(data_path)`: Reads the 2D CSV file and returns an $N \times 2$ `numpy` array. (3 marks)
2. `initialise_centers(data, K, init_centers=None)`: If `init_centers` is `None`, select K data points at random without replacement to serve as initial hub coordinates. Otherwise, convert and return the provided `init_centers`. (5 marks)
3. `initialise_labels(data)`: Returns a 1D zeros array of size N . (2 marks)
4. `calculate_distances(data, centers)`: Calculates the Euclidean distance from each of the N delivery points to each of the K hubs. Returns an $N \times K$ array. (*Hint: Reshape the spatial columns and use broadcasting!*) (8 marks)
5. `update_labels(distances)`: Assigns each delivery point to the nearest hub. Returns an array of size N . (4 marks)
6. `update_centers(data, labels, K)`: Updates hub coordinates to the mean of their assigned delivery points. (*Hint: Avoid looping over K by building a Boolean mask of shape (K, N) comparing `labels` to `np.arange(K)`, and taking the dot product (`@`) of this mask with the data matrix!*) (10 marks)
7. `check_termination(labels1, labels2)`: Returns `True` if no labels changed from the previous iteration. (2 marks)
8. `kmeans(data_path, K, init_centers)`: Ties the above functions together using a `while True:` loop. Tracks and returns the final `centers`, `labels`, and the `execution_time`. (6 marks)

Sample Input (`delivery_locations.txt`):

```
1.0, 1.0
2.0, 1.0
1.0, 2.0
8.0, 8.0
9.0, 8.0
8.0, 9.0
```

Sample Execution:

```
1 # Using deterministic initial centers for testing purposes
2 init_centers = [[1.0, 1.0], [8.0, 8.0]]
3 final_centers, labels, exec_time = kmeans('delivery_locations.txt', 2,
4     init_centers)
5 print("Final Centers:")
6 print(final_centers)
7 print("\nLabels:")
8 print(labels)
9 print(f"\nExecution Time: {exec_time:.4f} seconds")
```

Expected Output:

Final Centers:

```
[[1.33333333 1.33333333]
 [8.33333333 8.33333333]]
```

Labels:

```
[0 0 0 1 1 1]
```

Execution Time: 0.0015 seconds